

Power-of-Two Quantization-Aware-Training (PoT-QAT) in Large Language Models (LLMs)



Mahmoud Elgenedy

Computer Science, Stanford
melgened@stanford.edu

Motivation and Proposed Solution

Problem statement:

- In Large Language Models (LLMs), the number of parameters has grown exponentially in the past few years, e.g., from 1.5 billion parameters in GPT2 to 175 billion in GPT3.
- Challenging for implementation, considering Edge devices as memory and processing power are limited.

Proposed Solution:

- We propose reducing LLM complexity by compressing weights using special type of **quantization, power of two (PoT)**, where numbers are restricted to only power of two.
- PoT provides major **memory reduction** by reducing weights bit-widths, and more importantly provides significant computational reduction through converting **multiplication to bit-shifting**.
- Proposing additional training after quantization, i.e., Quantization Aware Training (**QAT**) to compensate performance degradation due to aggressive PoT quantization.
- PoT was investigated for LLM in [1] and [2] with post training quantization approach (**PTQ**), i.e., no additional training after quantization, and LLM general quantization with QAT was addressed in [3].
- To best of our knowledge, no clear studies investigated PoT quantization with QAT for LLMs.

Summary of Results

For **GPT-2 124M** model, the additional training (QAT) after PoT quantization successfully converges for different PoT settings, and is shown to significantly improve the performance degradation due to PoT quantization, and mostly closed the gap of quantized PoT model versus baseline full precision model,

- For PoT 15 levels, **QAT converges** after ≈ 455 steps, batch size 12, 64 block size.
- For PoT 15 levels, **perplexity after QAT** is enhanced from 90 (without additional training, PTQ) to **30** which is $\approx 66\%$ **improvement**, and very close to baseline full precision perplexity of 23.17.
- BERT-Score** is converging to baseline after 300 steps of QAT, with a **gap of only 1%**.
- The estimated **memory** reduction is **87.5 %** and **processing** speed up of a factor $\approx 3\text{-}10\times$.

LLM Model and Datasets

NanoGPT [4] is a repository for training and tuning Generative Pre-trained Transformer GPT. The design follows GPT-2 model and can reproduce GPT-2 on OpenWebText dataset [5]. Two models are considered,

1. Character based LLM with Shakespeare dataset

Our preliminary trials considering a character-level model trained on Shakespeare dataset (a small dataset (≈ 1 MB) consists of texts from William Shakespeare's plays.

2. GPT-2 124M model with OpenWebText dataset

We extended PoT-QAT results to a pretrained GPT-2 124M model [6] using OpenWebText dataset (relatively larger dataset ≈ 54 GB) for the additional QAT training and validation.

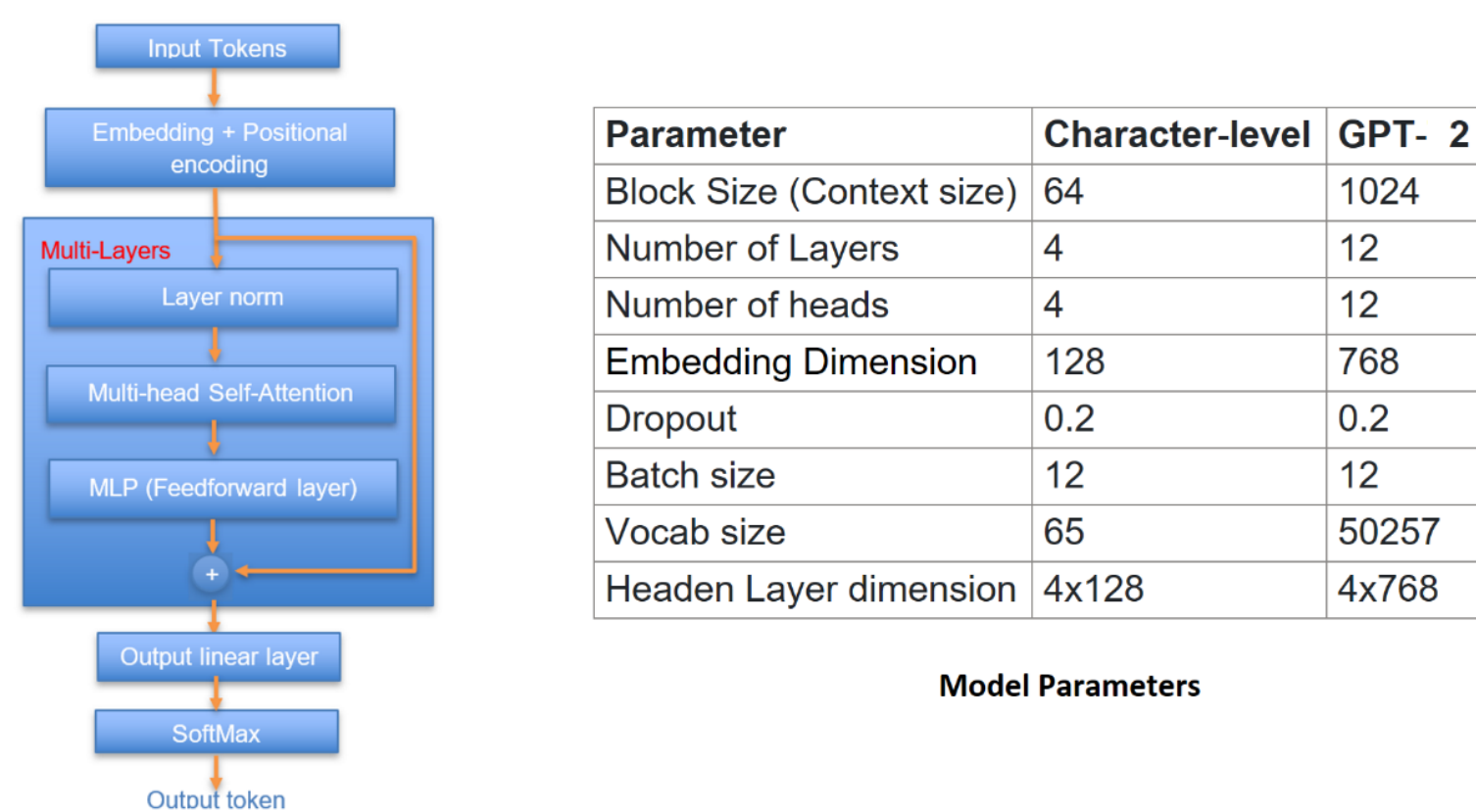


Figure 1. LLM Model illustration and parameters

Method

Quantization:

- Numbers are represented in lower precision to help reduce both memory and processing requirements.
- Different types including symmetric/asymmetric quantization and Power-of-Two (PoT) (Table 1).

Quantization type	Conversion equation (input x, output y)	Use case
Asymmetric (Affine)	$y = \text{round}(x / \text{scale}) + \text{zp}$	Activations
Power of Two (PoT)	$y = 2^{\text{clip}(\text{round}(\log_2(x / \text{scale})))}$	Suggested in this proposal for Weights

Table 1. Types of Quantization

Neural Networks Models Quantization Methods:

- Post Training Quantization (**PTQ**): model is trained in full precision floating point, then quantization is applied without additional training.
- Quantization Aware Training (**QAT**): Additional training is performed so that model can adapt to the quantization noise.

Power-of-Two Quantization-Aware-Training (PoT-QAT) challenge:

- QAT is a method that can theoretically apply to any quantization type.
- The main challenge is that training requires computing gradient. Quantization in general is non-differentiable, additionally, PoT has a non-uniform step size (uniform in log domain).
- We followed standard way to approximate the gradient to identity in the backward path, called Straight Through Estimator (**STE**), and it is proved that system is converging with this approximation.

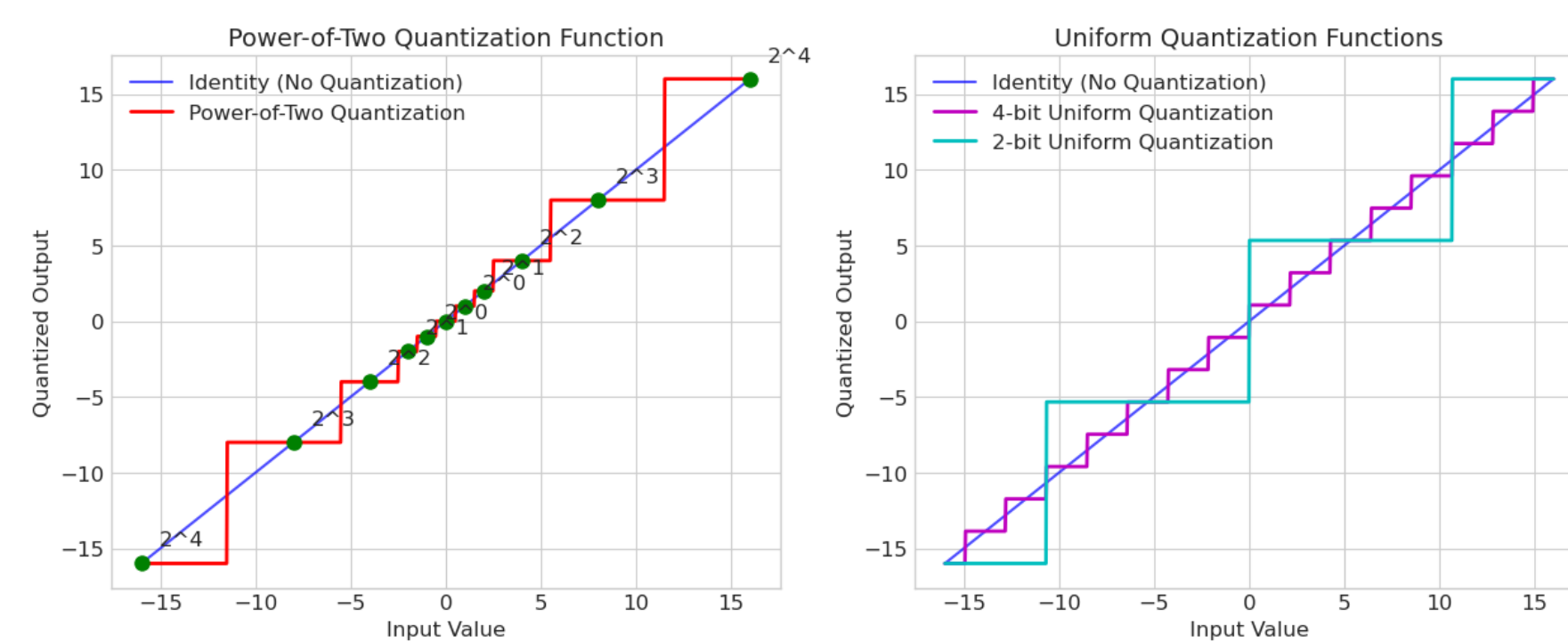


Figure 2. Power-of-Two (PoT) Versus Uniform Quantization

Quantization Aware Training (QAT) PyTorch Framework

We use PyTorch 2 Export Mode, where model is transformed into a graph of nodes (low level ops), and then fake quantization operations are inserted between the different nodes to simulate quantization noise.

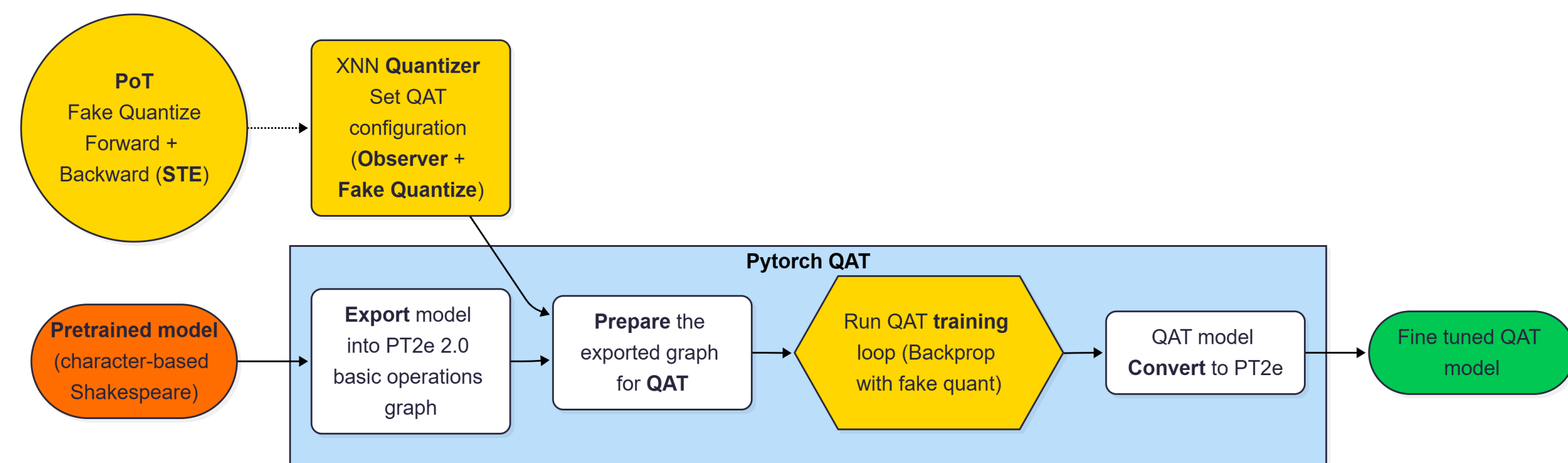


Figure 3. PyTorch Quantization Aware Training (QAT) Framework

Performance Evaluation & Main Results (GPT-2 124M)

Training/Validation Convergence: PoT for different configurations successfully converge, with best 15-levels (4-bits) reaches smallest loss (3.41) after ≈ 455 steps (a slight overfit after that). Other configurations (7 and 11 levels) show higher bias, but expect less error with training as no overfitting is observed.

Perplexity: QAT shows significant improvement for perplexity with training, with PoT-15 improved from 90 to 30 ($\approx 66\%$), which is very close to baseline full precision of 23.1.

BERT-Score: BERT-Score shows a clear convergence with training, where PoT-15 BERT-Score diff reaches 1% after 330 training steps. This is aligned with cross-entropy loss of 3.78, which is not the lowest validation error (3.41) indicating slight overfitting toward the training end.

Memory and Computational Savings: Memory reduction comes as benefit from smaller bit-width (87.5 % for 4 bits) while computational savings (3-10x reduction) come as a result of smaller bit-width and the power of two feature that replaces multiplication with bit-shifting.

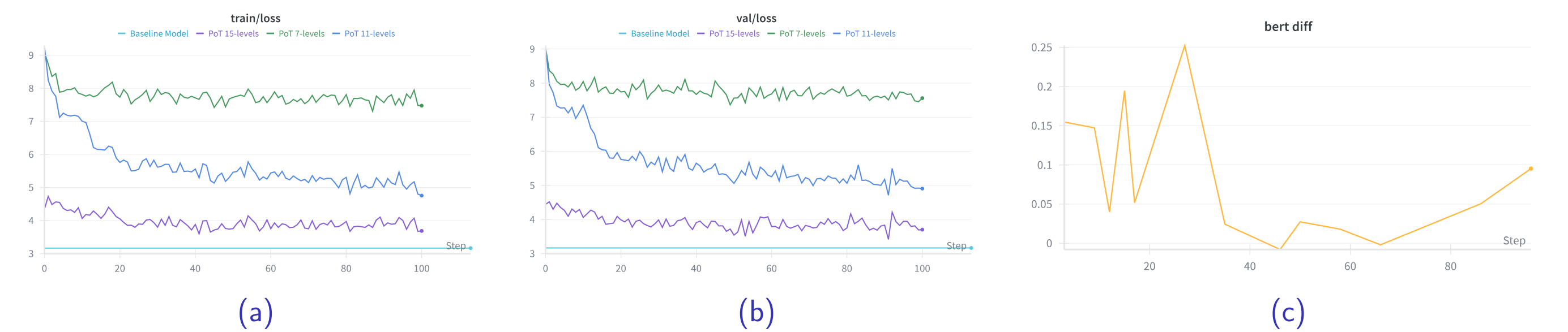


Figure 4. Quantization Aware Training convergence for different PoT settings. Each step is 5 training step iterations. (a) Training loss versus iterations. (b) Validation loss versus iterations. (c) BERT-Score delta for PoT-QAT quantized vs baseline floating-point, PoT-15 at val loss ≈ 3.78

quantization range		PoT - PTQ		PoT - QAT	
Integer range	N bits	l_{ce}	Perplexity	l_{ce}	Perplexity
Baseline float32	32 bits	3.167	23.73	3.167	23.167
$[-2^3, 2^3]$	3 bits	9.01	8184	7.42	1669
$[-2^7, 2^7]$	4 bits	4.5	90	3.417	30.47

Table 2. Perplexity of PoT post training quantization PTQ versus PoT quantization aware training quantization QAT

Metric	FP32	4-bit PoT
Model Size	124 MB x 4 = 496 MB	62 MB (87.5% saving)
Inference Speed	1x	3-10x faster

Table 3. GPT-2 124M: FP32 vs. 4-bit Power-of-Two Quantization

Conclusions & Future Work

Power-of-Two (PoT) quantization is very efficient quantization method that helps significantly reduce memory and computational cost. Additional training after quantization (QAT) is very powerful and proved to enhance the performance and close the gap of degradation due to PoT quantization.

Future work includes experimenting the PoT-QAT on bigger models like Llama and Qwen. Also, it will be very interesting to try the algorithm on device and measure the actual complexity saving.

References

- Elgenedy, Mahmoud. "Power-of-Two (PoT) Weights in Large Language Models (LLMs)." arXiv:2506.00315 (2025).
- Wang, Xinyu, et al. "PoTPTQ: A Two-step Power-of-Two Post-training for LLMs." arXiv preprint arXiv:2507.11959 (2025).
- Liu, Zechun, et al. "Llm-qat: Data-free quantization aware training for large language models." Findings of the Association for Computational Linguistics: ACL 2024. 2024.
- <https://github.com/karpathy/nanoGPT>, by Karpathy.
- <https://huggingface.co/datasets/Skylion007/openwebtext>.
- <https://huggingface.co/openai-community/gpt2>.